DCBD Midsem notes

Arushi Marwaha

March 2026

Contents

1 Distributed vs Decentralized Systems

1.1 Definitions

Decentralized System

A **decentralized system** is a networked computer system in which processes and resources are *necessarily* spread across multiple computers. There is no single central authority controlling the system, and each participating node may operate independently.

Distributed System

A **distributed system** is a networked computer system in which processes and resources are *sufficiently* spread across multiple computers so that they work together to appear as a single coherent system to the user.

Key Idea: All distributed systems involve multiple machines, but not all decentralized systems aim to behave like a single unified system.

1.2 Main Differences

- **Decentralized systems** focus on removing central control.
- **Distributed systems** focus on coordinating multiple machines to act as one logical system.
- In distributed systems, components are tightly coordinated.
- In decentralized systems, components often operate more independently.

1.3 Real-Life Examples

Distributed System Example

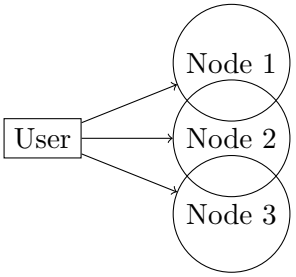
Cloud platforms such as **Google Search** or **Netflix** run on thousands of machines but appear as a *single service* to users. The infrastructure is distributed, yet the system behaves like one unified platform.

Decentralized System Example

Bitcoin or blockchain networks are decentralized. There is no central server, and each node independently participates in maintaining the system.

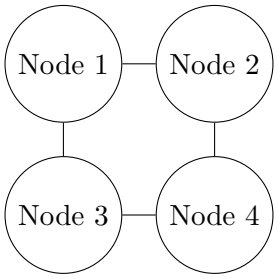
1.4 Illustration

Distributed System



Here multiple nodes cooperate to provide a single service.

Decentralized System



In decentralized systems, nodes communicate directly without relying on a central controller.

2 Processes

2.1 What is a Process?

A **process** is an instance of a computer program that is currently being executed.

A **program** is passive: it is simply a file containing instructions stored on disk. A **process** is active: the program is loaded into memory and executed by the CPU.

Key Idea: A program becomes a process when the operating system loads it into memory and begins execution.

2.2 Resources of a Process

When a program runs as a process, the operating system allocates several memory regions:

- **Text Segment** – Contains the compiled machine instructions of the program.
- **Data Segment** – Stores global, static, and constant variables.
- **Heap** – Used for dynamic memory allocation during runtime (e.g., using `malloc` or `new`).
- **Stack** – Stores temporary data such as local variables, function parameters, return addresses, and function call information.

Key Idea: The stack grows and shrinks during function calls, while the heap grows dynamically as memory is allocated.

2.3 Example

Consider a Python program:

```
def add(a, b):
    return a + b

print(add(3,4))
```

When executed:

- The compiled instructions are stored in the **text segment**.
- Global variables go into the **data segment**.
- Function calls such as `add(3,4)` create stack frames in the **stack**.
- Any dynamic objects created during execution are stored in the **heap**.

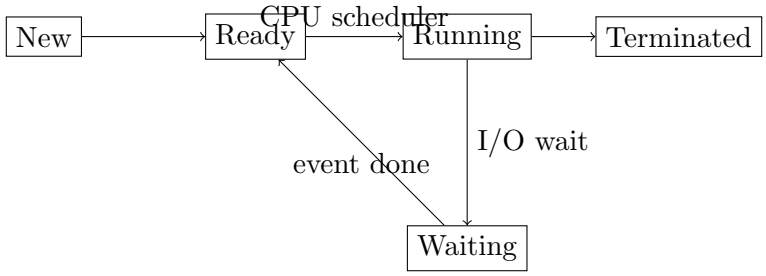
2.4 Process Life Cycle

A process moves through different states during its execution.

- **New (Start)** – The process is created by the operating system.
- **Ready** – The process is loaded into memory and waiting for CPU allocation.
- **Running** – The process is currently being executed by the CPU.
- **Waiting (Blocked)** – The process is waiting for an external event such as I/O completion.
- **Terminated (Exit)** – The process has finished execution or has been stopped by the operating system.

Important: At any moment, only one process can be running on a CPU core.

2.5 Process State Diagram



2.6 Real-World Example

When you open **Google Chrome**:

- The Chrome program (stored on disk) is loaded into memory.
- The operating system creates a **process**.
- The process moves to the **ready** state waiting for CPU time.
- It enters the **running** state when the CPU executes it.
- If the browser loads a webpage, it may enter the **waiting** state while waiting for network data.
- When Chrome is closed, the process enters the **terminated** state.

3 Orphan and Zombie Processes

3.1 Orphan Process

An **orphan process** is a process whose parent process has terminated while the child process is still running.

Normally, processes are created in a *parent-child hierarchy*. When a parent process terminates before its child, the child becomes an orphan.

In most operating systems (e.g., Linux), orphan processes are automatically adopted by a special system process called `init` (or `systemd`), which becomes the new parent.

Example

Suppose a terminal launches a program that creates a child process:

- Parent process terminates.
- Child process continues running.
- The OS reassigns the child process to `init`.

Key Idea: Orphan processes are still running processes but without their original parent.

3.2 Zombie Process

A **zombie process** is a process that has finished execution but still has an entry in the **process table**.

When a child process terminates, it sends its **exit status** to the parent. The parent must read this status using system calls such as `wait()`.

If the parent has not yet read the exit status, the process remains in the process table as a zombie.

Key Idea: A zombie process is already dead but still occupies a process table entry.

Example

- Child process finishes execution.
- Parent process does not call `wait()`.
- The child becomes a zombie until the parent collects its exit status.

3.3 Thread

A **thread** is an independent flow of control within a process. Multiple threads can exist inside the same process and share the same memory space.

Threads allow multiple tasks to execute concurrently within a single program.

Key Idea: Threads share memory, whereas processes have separate memory spaces.

3.4 Process vs Thread

Feature	**Process**	**Thread**
Definition	Independent program in execution	Unit of execution within a process
Memory Space	Separate memory space for each process	All threads share the same memory space
Weight	Heavyweight (more resources required)	Lightweight (less overhead)
OS Interaction	Context switching requires OS intervention	Switching is faster and lighter
Data Visibility	Data is isolated from other processes	Threads can directly access shared data
Communication	Uses IPC mechanisms (pipes, sockets, etc.)	Direct communication via shared memory

3.5 Multithreading

Multithreading allows multiple threads to run within a single process.

This improves CPU utilisation by allowing different tasks to progress concurrently.

Key Features

- **Shared Resources:** Threads share variables and memory.
- **Fast Creation:** Creating threads is faster than creating processes.
- **Lightweight:** Requires less memory and overhead.
- **Risk:** Shared memory can lead to **race conditions**.

Example

A web browser may use multiple threads:

- One thread loads web content

- Another handles user interface
- Another plays video/audio

3.6 Concurrency vs Parallelism

Concurrency

Concurrency means multiple tasks are **in progress at the same time**. Tasks are interleaved by the CPU scheduler.

This often occurs on a **single CPU core**.

Example

While a program waits for network data, the CPU switches to another task.

Parallelism

Parallelism means multiple tasks are executed **simultaneously** on different CPU cores.

Example

Four CPU cores processing four different images at the same time.

3.7 Multiprocessing

Multiprocessing refers to using multiple CPUs or cores to execute processes simultaneously.

Each process runs in its own memory space.

Working of a Multiprocessing System

1. Multiple CPUs share main memory.
2. The OS divides the workload into processes.
3. Processes are scheduled across CPUs.
4. Each CPU executes its assigned task in parallel.
5. Results are combined if required.

Advantages

- High reliability (process isolation)
- True parallel execution
- Improved throughput

3.8 Global Interpreter Lock (GIL)

The **Global Interpreter Lock (GIL)** is a mutual exclusion lock used in the CPython interpreter.

It ensures that only **one thread executes Python bytecode at a time**.

This means:

- Python supports multithreading
- But CPU-bound threads cannot run in parallel

However, threads release the GIL during **I/O operations**, so multithreading works well for I/O-bound tasks.

3.9 Multithreading vs Multiprocessing

Feature	Multithreading	Multiprocessing
Best For	I/O-bound tasks	CPU-bound tasks
Execution	Concurrency	True parallelism
Memory	Shared memory space	Separate memory space
Overhead	Low	High
Python Limitation	Affected by GIL	Bypasses GIL

3.10 When to Use Multiprocessing

- CPU-intensive computations
- Machine learning model training
- Large data processing
- Image or video processing

Example
Converting 1000 high-resolution images into JPEG format. Each process can convert a different image independently.

3.11 When to Use Multithreading

- Network requests
- File reading and writing
- Web scraping
- User interface responsiveness

Example
In a browser:

- One thread downloads files
- Another keeps the interface responsive
- Another renders the page

Key Exam Insight

- **CPU-bound tasks → Multiprocessing**
- **I/O-bound tasks → Multithreading**

4 Virtualization

4.1 Definition

Virtualization is a technology that allows multiple virtual computing environments to run on a single physical machine. It works by creating an **abstraction layer** over the physical hardware so that multiple operating systems and applications can share the same resources efficiently.

Key Idea: Virtualization separates *hardware* from *software environments*, allowing multiple virtual systems to run on the same machine.

4.2 Why Virtualization is Important

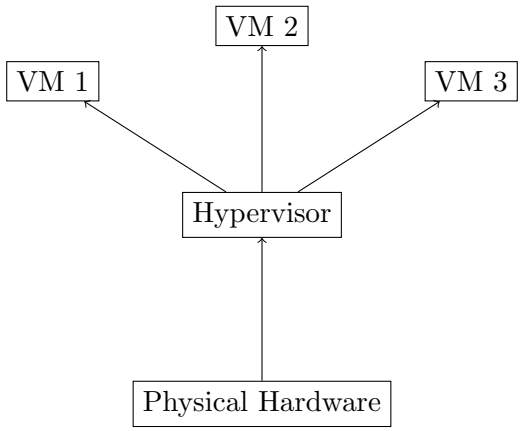
- **Resource Efficiency:** Many servers typically use only about 10% of their capacity. Virtualization allows better utilisation of hardware.
- **Isolation:** Each virtual machine runs independently, creating secure sandboxes for testing or development.
- **Encapsulation:** A complete operating system can be stored as a single file and easily moved to another machine.
- **Cloud Foundation:** Modern cloud platforms such as AWS, Azure, and Google Cloud rely heavily on virtualization.

4.3 How Virtualization Works

Virtualization creates a software layer that divides physical resources such as:

- CPU
- Memory
- Storage
- Network

into multiple **virtual machines (VMs)** that behave like independent computers.



4.4 Benefits of Virtualization

Problem	**Physical-Only Approach**	**Virtualized Approach**
Costs	Need to buy more hardware	Use existing hardware efficiently
Setup Time	Days or weeks	Minutes
Redundancy	Buy duplicate machines	Move VMs between machines
Environmental Impact	High power and cooling use	Energy efficient

4.5 Virtual Machine (VM)

A **Virtual Machine (VM)** is a software-based representation of a physical computer.
It runs its own:

- Operating system
- Applications
- Memory and CPU allocation

The physical machine is called the **host**, and the virtual machines running on it are called **guest machines**. Multiple VMs can run simultaneously on one physical computer.

4.6 Hypervisor

A **hypervisor** is the software layer that manages virtual machines. It allocates resources and ensures that VMs do not interfere with each other.

Key Role of Hypervisor

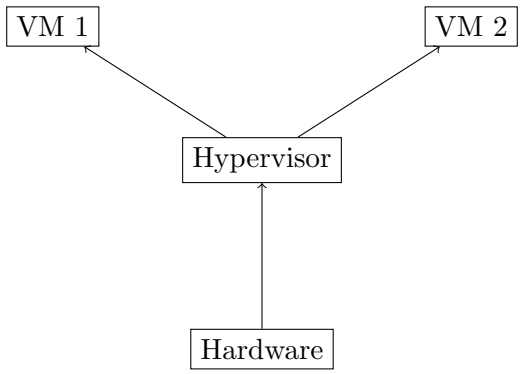
- Creates and manages VMs
- Allocates CPU, memory, and storage
- Ensures isolation between VMs

4.7 Types of Hypervisors

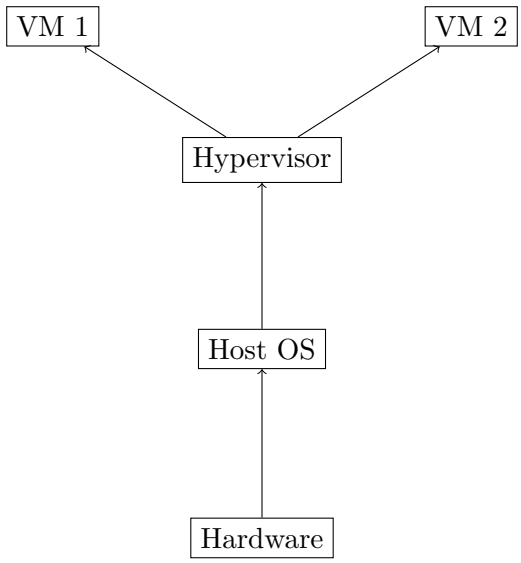
Feature	**Type 1 (Bare-Metal)**	**Type 2 (Hosted)**
Location	Runs directly on hardware	Runs on top of a host OS
Efficiency	High efficiency	Moderate efficiency
Use Case	Data centers and servers	Personal machines and development
Stability	Very high	Depends on host OS stability

4.8 Architecture

Type 1 Hypervisor



Type 2 Hypervisor



4.9 Real-World Example

Cloud providers use virtualization extensively.

For example, when you launch a server on **AWS EC2**:

- You are actually launching a **virtual machine**.
- Many VMs run on the same physical server.
- Each VM behaves like an independent computer.

Key Exam Insight:

- Virtualization enables **resource sharing and isolation**.
- It forms the **foundation of modern cloud computing**.

4.10 Case Study: Choosing the Right Hypervisor

Scenario

A company wants to deploy multiple applications on shared hardware. They need to decide between using a **Type 1 (Bare-Metal)** hypervisor or a **Type 2 (Hosted)** hypervisor.

4.10.1 Case 1: Cloud Data Center

A cloud provider such as Amazon Web Services needs to run thousands of virtual machines on powerful physical servers. Each VM belongs to a different customer and must be isolated and secure.

Solution

A **Type 1 hypervisor** is used because it runs directly on the hardware and provides high performance and strong isolation.

Examples

- VMware ESXi
- Microsoft Hyper-V
- Xen

Why it is suitable

- Direct access to hardware
- Low overhead
- High scalability
- Strong security between virtual machines

Real Example
When a user launches an EC2 instance on AWS, it is actually a virtual machine running on a physical server managed by a Type 1 hypervisor.

4.10.2 Case 2: Developer Testing Environment

A software developer wants to test their application on multiple operating systems such as Windows, Linux, and Ubuntu using their personal laptop.

Solution
A **Type 2 hypervisor** is used because it runs on top of the existing operating system and is easy to install.
Examples

- VirtualBox
- VMware Workstation
- Parallels Desktop

Why it is suitable

- Easy to install on personal machines
- No need to manage physical servers
- Useful for testing and development

Real Example
A developer running Ubuntu inside VirtualBox on a Windows laptop to test Linux-specific software.

4.11 Key Exam Insight

- **Type 1 Hypervisor:** Used in cloud platforms and data centers where performance and scalability are critical.
- **Type 2 Hypervisor:** Used in personal computers for development, testing, and experimentation.

4.12 Security and Attack Surface in Virtualization

Key Idea: The security of a virtualized system depends on the number of **attach points** (also called the **attack surface**). An attach point is any location where an unauthorized user could potentially interact with or exploit the system.

4.13 Type 1 Hypervisor Security

In a **Type 1 (Bare-Metal) hypervisor**, the hypervisor runs directly on the hardware.

- Fewer software layers
- Smaller attack surface
- Higher security and performance

Because there are fewer components between the hardware and the virtual machines, attackers have fewer entry points.

Example

Cloud providers such as AWS or Azure use Type 1 hypervisors because they provide stronger isolation between customer virtual machines.

4.14 Type 2 Hypervisor Security

In a **Type 2 (Hosted) hypervisor**, the hypervisor runs on top of a host operating system.

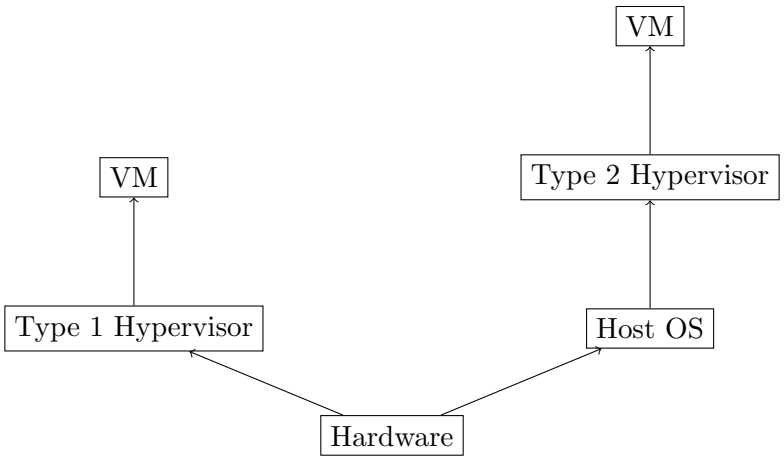
- Additional software layer (Host OS)
- Larger attack surface
- More potential vulnerabilities

If the host operating system is compromised, the attacker may gain access to all virtual machines running on top of it.

Example

A developer using VirtualBox on a laptop depends on the security of the host OS (Windows/macOS/Linux). If malware infects the host OS, the virtual machines may also be at risk.

4.15 Conceptual Illustration



4.16 Exam Insight

- **Security depends on attack surface (number of attach points).**
- **Type 1 hypervisors** have fewer layers and therefore fewer vulnerabilities.
- **Type 2 hypervisors** introduce additional risk because the host OS becomes part of the trusted computing base.

4.17 Resource Partitioning

In a virtualized system, the **hypervisor** is responsible for dividing the physical hardware resources among multiple virtual machines (VMs). It acts like an **accountant or resource manager**, allocating CPU, memory, storage, and network bandwidth to each VM.

Example

Suppose a physical server has:

- 16 GB RAM
- 4 CPU cores

The hypervisor can partition these resources into multiple virtual machines, for example:

- VM 1: 4 GB RAM + 1 vCPU
- VM 2: 4 GB RAM + 1 vCPU
- VM 3: 4 GB RAM + 1 vCPU
- VM 4: 4 GB RAM + 1 vCPU

Each VM behaves like an independent computer.

Key Idea: The hypervisor ensures **resource isolation**. Even if one VM crashes or tries to consume excessive resources, the other VMs remain stable.

4.18 Snapshots

A **snapshot** is a mechanism that captures the complete state of a virtual machine at a specific moment in time.

When a snapshot is created:

- The hypervisor freezes the current state of the virtual disk.
- A **delta file** is created to record all future changes.

The original state remains preserved, allowing the system to return to it later.

Benefit

If something breaks in the system (for example, after installing faulty software), the VM can quickly **revert to the snapshot** instead of reinstalling the entire operating system.

4.18.1 Real-World Example

Suppose a developer wants to test a risky update on a Linux virtual machine.

1. The developer creates a snapshot before installing the update.
2. The update causes the system to crash.
3. Instead of reinstalling Linux, the developer simply restores the snapshot.

Within seconds, the VM returns to its previous working state.

Key Exam Insight

- **Resource Partitioning:** Hypervisor divides hardware resources among multiple VMs.
- **Snapshots:** Allow fast rollback to a previous system state.

5 Containers and Containerization

5.1 Container

A **container** is a standardized unit of software that packages an application together with all of its dependencies so that it runs reliably across different computing environments.

A container includes:

- Application code
- Runtime environment
- System tools
- Libraries
- Configuration files

Key Idea: Containers isolate applications from the underlying environment so that the software behaves the same in development, testing, and production.

5.2 Container Image

A **container image** is a lightweight, standalone, executable package that contains everything required to run an application.

- Images are **read-only**.
- Images act as a **blueprint** for containers.
- Containers are the **running instances** of images.

Example

An image such as `python:3.9-slim` contains the Python runtime and system libraries. When executed, it creates a container that runs a Python application.

5.3 Containerization

Containerization is the process of packaging an application together with all its dependencies so that it can run consistently on any infrastructure.

Example

A web application developed on a developer's laptop can be packaged into a container and run on:

- local machine
- testing server
- cloud infrastructure

without changing the code.

5.4 Virtual Environment vs Container

Aspect	Virtual Environment (venv)	Container (Docker)
Goal	Isolate Python libraries	Isolate entire application environment
Isolation Level	Python packages only	Application + system libraries + binaries
OS Usage	Uses host OS directly	Shares kernel but isolates filesystem
Portability	Low (may break across systems)	High (runs the same everywhere)
Performance	Native speed	Near-native performance
Dependency Conflicts	Solves Python conflicts only	Solves system-level conflicts
Deployment	Manual setup required	Standardized deployment with Docker

5.5 Container Runtime

A **container runtime** is the software responsible for running containers and managing their lifecycle.

Responsibilities include:

- starting and stopping containers
- managing container images
- interfacing with the host operating system

Examples

- Docker
- Podman
- containerd

5.6 Container Platforms

Several tools are used for container management in distributed systems.

- **Docker** – Platform for building and running containers.
- **Kubernetes** – Orchestrates and manages large numbers of containers.
- **Docker Swarm** – Docker's native clustering solution.
- **Apache Mesos** – Manages containerized and non-containerized workloads.

5.7 Replication and Installation

5.7.1 Container Engine

Containers are not installed like traditional applications. Instead, a **container engine** (e.g., Docker) is installed.

The engine:

- manages containers

- communicates with the Linux kernel
- translates container requests into system operations

5.7.2 Image vs Container

Feature	Image	Container
Definition	Blueprint of application	Running instance of image
State	Static / immutable	Dynamic / temporary
Purpose	Build and distribution	Execution of application
Example	nginx:latest	Running nginx web server

5.8 Orchestration

When hundreds or thousands of containers run across many machines, orchestration tools are required.
Orchestration is the automated coordination and management of containerized applications.
Main Responsibilities

- Deploy containers
- Scale applications
- Monitor health of services
- Replace failed containers

5.9 Kubernetes

Kubernetes (K8s) is the most widely used container orchestration platform.
Key features:

- **Automatic Scaling** – Creates more container instances during traffic spikes.
- **Self-Healing** – Automatically restarts failed containers.
- **Load Balancing** – Distributes requests across containers.

5.10 Example of Container Orchestration

Consider an e-commerce website deployed using containers.

1. Each service (frontend, payment, database) runs inside containers.
2. Kubernetes monitors system load.
3. If traffic increases, Kubernetes automatically creates more containers.
4. If a container crashes, Kubernetes replaces it automatically.

Key Exam Insight

- **Image = Blueprint**
- **Container = Running instance**
- **Orchestration = Automated management of containers**

6 Addressing Distributed System Fallacies with Containers

In distributed systems, developers often make incorrect assumptions known as the **Fallacies of Distributed Computing**. Modern container platforms and orchestration tools provide patterns and mechanisms to mitigate these fallacies.

6.1 Fallacy: The Network is Reliable

In reality, networks fail due to timeouts, packet loss, and service outages.

Solution: Sidecar Pattern

A **sidecar container** runs alongside the main application container and handles networking concerns such as:

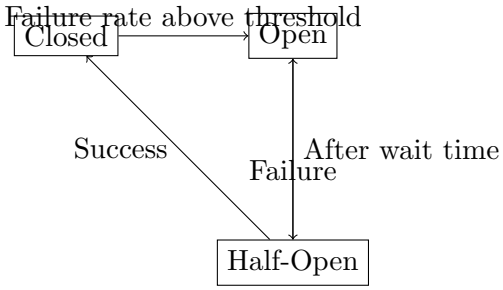
- retries
- failover logic
- circuit breakers
- service communication

This allows the main application to remain simple while the sidecar manages network resilience.

6.2 Circuit Breaker Pattern

A **circuit breaker** is a design pattern that prevents a system from repeatedly trying to execute an operation that is likely to fail.

Key Idea: If failures exceed a threshold, the system stops sending requests temporarily to avoid cascading failures.



States

- **Closed:** Requests are sent normally.
- **Open:** Requests are blocked to prevent further failures.
- **Half-Open:** A few test requests are allowed to check if the system has recovered.

Example

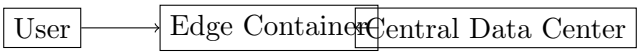
If a payment service becomes unavailable, the circuit breaker stops repeated calls to that service until it recovers.

6.3 Fallacy: Latency is Zero

Network communication introduces delays.

Solution: Edge Deployment

Containers can be deployed closer to users using **edge computing**. This reduces latency by processing requests near the user instead of sending all traffic to a distant central data center.



Example

Content delivery networks (CDNs) deploy services near users to reduce response time.

6.4 Fallacy: Bandwidth is Infinite

Network bandwidth is limited, so transferring large container images can be slow.

Solution: Layered Images

Container images are built in layers:

- Base OS layer
- Dependency layer
- Application code layer

When updates occur, only the changed layer is downloaded instead of the entire image.

Example

If only the application code changes, the base OS and dependency layers are reused.

6.5 Fallacy: The Network is Secure

Distributed systems cannot assume that networks are secure.

Container Security Strategy

Instead of patching compromised containers, orchestration systems typically:

- terminate the compromised container
- replace it with a fresh instance

Key Idea: Containers are designed to be **disposable**. If one becomes compromised or corrupted, it is destroyed and recreated from the original image.

6.6 Real-World Example

Consider a large microservices application deployed using Kubernetes:

- Sidecar containers manage retries and circuit breakers.
- Edge deployments reduce latency for global users.
- Layered images reduce bandwidth usage during updates.
- If a container is compromised, Kubernetes automatically replaces it.

Exam Insight

Container platforms help address distributed system fallacies by improving:

- fault tolerance

- scalability
- security
- deployment efficiency

6.7 Addressing More Fallacies of Distributed Computing

Distributed systems must handle incorrect assumptions about how networks and systems behave. Container platforms and orchestration systems (such as Kubernetes) provide mechanisms to mitigate these fallacies.

6.7.1 Fallacy: Topology Does Not Change

In distributed environments, machines and containers frequently start, stop, or change IP addresses.

Solution: Service Discovery

Service discovery allows containers and services to find each other dynamically without relying on fixed IP addresses.

- Orchestrators assume that the system topology constantly changes.
- Containers register themselves with a discovery service.
- Other containers query the service to locate them.

Example

In Kubernetes, services are accessed using logical names rather than IP addresses. Even if a container restarts with a new IP address, other services can still locate it using the service name.

6.7.2 Fallacy: There Is One Administrator

In modern distributed systems, multiple teams manage different parts of the infrastructure.

Solution: Decoupling (Separation of Concerns)

Container-based systems separate responsibilities between developers and operators.

- **Developers** build container images with the application and dependencies.
- **Operators** deploy and manage these containers in production environments.

Example

A developer builds a Docker image for a web application, while the operations team deploys it using Kubernetes.

6.7.3 Fallacy: Transport Cost Is Zero

Network communication is expensive in terms of latency, bandwidth, and reliability.

Solution: Efficient Transport Protocols

Distributed systems use optimized protocols to reduce overhead and improve communication efficiency.

- Lightweight communication protocols
- Efficient serialization formats
- Optimized service-to-service communication

Example

Microservices often use efficient RPC protocols such as `gRPC` instead of heavier protocols to reduce transport cost.

6.7.4 Fallacy: The Network Is Homogeneous

Real-world distributed systems run on diverse hardware, operating systems, and environments.

Solution: Container Standardization

Containers provide a consistent execution environment across different infrastructures.

- Applications run the same way on any host system.
- Differences in operating systems and environments are abstracted.

Example

A container built on a developer's laptop can run identically on a cloud server or data center machine.

6.7.5 Security Handling in Container Systems

When containers become compromised or unstable, modern systems do not attempt to repair them.

- Compromised containers are terminated.
- New containers are created from trusted images.

Key Idea: Containers are designed to be **ephemeral**. Instead of patching a broken instance, the system replaces it with a fresh one.

6.7.6 Exam Insight

Container orchestration platforms address several distributed system fallacies through:

- dynamic service discovery
- separation of responsibilities
- efficient network communication
- standardized execution environments
- automatic replacement of failed containers

6.8 REST API vs gRPC

REST and **gRPC** are two widely used communication mechanisms for services in distributed systems.

Feature	**REST API**	**gRPC**
Data Format	JSON (text-based, heavier)	Protocol Buffers (binary, compact)
Protocol	HTTP/1.1	HTTP/2
Speed	Slower because JSON parsing is CPU intensive	Faster due to binary serialization and efficient streaming
Code Generation	Client must manually parse responses	Client/server code auto-generated from `.proto` files
Browser Support	Excellent, supported directly by browsers	Poor, usually requires a proxy (e.g., gRPC-Web)
Best Use Case	Public APIs and web services	Internal microservices communication

Example

- A public web API (e.g., Twitter API) typically uses **REST + JSON**.
- Communication between microservices inside a company (e.g., service A calling service B) often uses **gRPC** for better performance.

Key Exam Insight

- REST is easier and widely supported.
- gRPC is faster and better for internal service-to-service communication.

6.9 Decorator Pattern vs Sidecar Pattern

These patterns are used to extend the functionality of applications in distributed systems.

Feature	Decorator Pattern	Sidecar Pattern
Location	Inside the application code	Outside the application (separate container/service)
Coupling	High: tightly coupled to code and language	Low: loosely coupled, communicates via network
Language Dependency	Must match the application language	Language-agnostic
Isolation	None. If decorator crashes, application crashes	High. Sidecar failure may not crash the main app
Updates	Requires recompiling and redeploying the app	Can be updated independently
Latency	Near zero (function call in memory)	Slightly higher (local network call)

Decorator Pattern

The decorator pattern adds functionality to a program by modifying or wrapping functions inside the application code.

Example

Adding logging or authentication inside a service function.

Sidecar Pattern

The sidecar pattern runs a helper component alongside the main service (often in another container).
It handles tasks such as:

- logging
- monitoring
- retries
- circuit breakers

Example

In Kubernetes, a service might have:

- Main container → application logic
- Sidecar container → networking or monitoring

Key Exam Insight

- Decorator modifies application code.
- Sidecar adds functionality externally without modifying the application.

7 Procedure Calls and Remote Procedure Calls (RPC)

7.1 Local Procedure Call (LPC)

A **Local Procedure Call (LPC)** is a function call where both the caller and the called procedure execute on the **same machine and within the same process address space**.

When a function is called locally:

- Control transfers from the calling function to the called procedure.
- The **program counter jumps** to the procedure's instructions.
- Arguments are passed through the stack or registers.
- After execution, control returns to the calling function.

Key Idea: Since both procedures run in the same address space, they can directly access shared memory such as the **stack and heap**.

7.2 Why RPC is Needed

In distributed systems, services often run on **different machines**. A normal function call cannot be used because:

- Processes do not share the same memory space.
- Data must be transmitted over the network.
- Failures and latency may occur.

To solve this, distributed systems use **Remote Procedure Calls (RPC)**.

7.3 Remote Procedure Call (RPC)

A **Remote Procedure Call (RPC)** allows a program to execute a procedure on a **remote machine** as if it were a local function call.

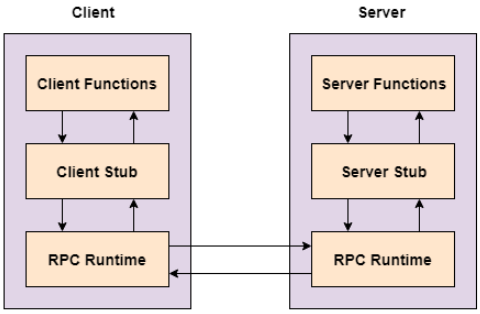
RPC hides network communication behind a familiar function call abstraction.

Key Idea: RPC makes remote service communication look like a local function invocation.

7.4 RPC Workflow

1. The client calls a local function stub.
2. The stub **serializes (marshals)** the parameters.
3. The request is sent over the network.
4. The server receives and **deserializes** the parameters.
5. The server procedure executes.
6. The result is sent back to the client.

7.4.1 RPC Architecture



7.5 The Middleware Layer

RPC is part of the **middleware layer** in distributed systems.

Middleware sits between:

- the **host operating system**
- and higher-level orchestration platforms (e.g., Kubernetes)

It acts as the **communication fabric** that allows distributed services to interact seamlessly.

7.6 The Three Pillars of Middleware

1. **Remote Procedure Calls (RPC)** Enables communication between distributed services.
2. **Temporal Synchronization** Ensures correct ordering of events in distributed systems.
3. **Service Discovery** Maps logical service names to dynamic endpoints (e.g., containers with changing IPs).

7.7 Real-World Example

In a microservices architecture:

- A **User Service** calls the **Payment Service**.
- The services run on different machines.
- RPC frameworks such as **gRPC** allow the call to look like a normal function call.

Exam Insight

- LPC: Function calls within the same machine and memory space.
- RPC: Function calls executed on remote machines through the network.
- Middleware enables communication, synchronization, and discovery in distributed systems.

8 Case Study: Swiggy and MyGate Interaction Using RPC

Consider a scenario where a delivery agent from Swiggy arrives at a gated residential society that uses the MyGate security system.

When the delivery agent reaches the gate, the Swiggy backend must communicate with the MyGate backend to request entry permission from the resident.

8.1 System Components

- Swiggy Backend Server
- MyGate Backend Server
- Resident Mobile Application
- Delivery Agent Application

These components run on different machines and communicate over the internet.

8.2 Sequence of Events

1. The delivery agent arrives near the society gate.
2. The Swiggy mobile application detects the location and sends a request to the Swiggy backend.
3. The Swiggy backend performs a Remote Procedure Call (RPC) to the MyGate backend requesting entry authorization.
4. The MyGate backend logs the request and sends a notification to the resident's mobile application.
5. The resident approves or denies the request.
6. The MyGate backend sends the response back to the Swiggy backend via RPC.
7. The Swiggy backend updates the delivery agent application with the result.

8.3 Challenges in Distributed Systems

Several issues may arise in such distributed interactions:

- **Network Failures:** The RPC request or response may be lost due to network issues.
- **Duplicate Requests:** If Swiggy retries a failed RPC, the system must ensure that duplicate gate requests are not created.
- **Latency:** Communication between the two systems may introduce delays.
- **System Failures:** If the MyGate server crashes after receiving the request, the request should not be lost.

8.4 Solutions

- **Write-Ahead Logging (WAL):** MyGate logs the request before processing to ensure durability.
- **Idempotent RPC Calls:** Repeated requests should not create multiple entry permissions.
- **Observability:** Logs, metrics, and distributed tracing help engineers debug failures across systems.

8.5 Foundations of Remote Procedure Calls (RPC)

8.5.1 Goal: Syntactic Transparency

The primary goal of RPC is **syntactic transparency**. This means that invoking a procedure on a remote machine should look exactly like calling a local function from the programmer's perspective.

Key Idea: The programmer writes code as if the function were local, while the RPC system handles networking, serialization, and communication behind the scenes.

Example

```
result = add(5,3)   # Looks like a local function
```

In reality, the function `add()` might be running on a completely different machine.

8.5.2 Core Mechanisms of RPC

RPC relies on two main components:

- **Client Stub** – Acts as a local proxy for the remote service.
- **Server Skeleton** – Receives the request and invokes the actual procedure.

These components hide network communication from the application developer.

8.6 RPC Components: Stubs and Skeletons

8.6.1 Client-Side Stub

The **client stub** is a proxy object that resides in the client's address space. It exposes the same interface as the remote procedure so that the call appears local.

Responsibilities

- Accept parameters from the client program
- Perform **marshaling** (serialization of arguments)
- Send the request to the remote server
- Receive the response and return it to the client

Example
In Python, RPC calls are often handled by libraries such as:

- `xmlrpc.client`
- `gRPC`

These libraries automatically generate the client stub code.

8.6.2 Server-Side Skeleton

The **server skeleton** acts as the adapter on the server side.

It listens for incoming RPC requests and connects them to the actual implementation of the remote procedure.

Responsibilities

- Receive network requests

- **Unmarshal** (deserialize) the arguments
- Call the appropriate remote procedure
- Capture the return value
- Send the result back to the client

8.7 Real-World Example

Consider a distributed microservices application:

- A **User Service** needs to retrieve payment information.
- The client application calls a function such as `getPaymentDetails(userID)`.
- The RPC framework sends this request to the **Payment Service**.
- The server processes the request and returns the result.

To the programmer, it appears as a simple function call even though the services are running on different machines.

8.8 Exam Insight

- **Syntactic Transparency**: Remote calls look like local calls.
- **Client Stub**: Marshals arguments and sends request.
- **Server Skeleton**: Unmarshals arguments and executes procedure.
- RPC simplifies communication between distributed services.

8.9 Intuition: Why RPC is Needed in Distributed Systems

On a single computer, calling a function feels instantaneous:

```
result = add(5,3)
```

The program simply jumps to another part of memory, executes the function, and returns the result. Both the caller and the function share the same **process address space**.

However, in **distributed systems**, services often run on different machines. A function call may require sending data across a network to another computer.

Key Idea: A remote function call is actually a **network message exchange disguised as a normal function call**.

8.9.1 Real-World Example: Food Delivery Application

Consider what happens when a user presses **"Place Order"** on a food delivery application.

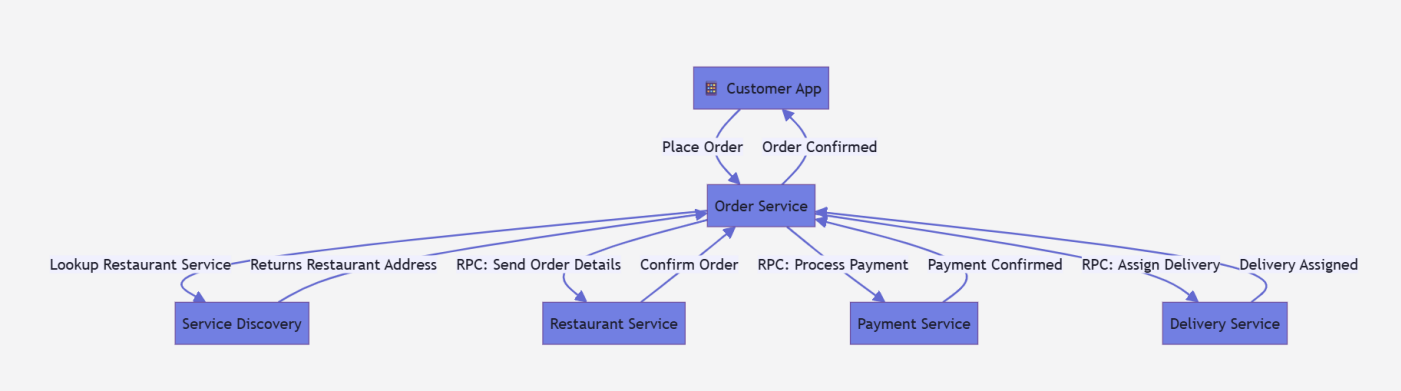
The request does not go to a single computer. Instead, it interacts with multiple specialized services:

- Order Service
- Restaurant Inventory Service
- Payment Service
- Delivery Assignment Service

Each service may run on a different machine in a distributed data center.
For example:

1. The user places an order.
2. The Order Service contacts the Restaurant Service to verify inventory.
3. The Payment Service processes payment.
4. The Delivery Service assigns a driver.

These communications between services are handled using **Remote Procedure Calls (RPC)**.



8.10 The Middleware Layer

Between the operating system and higher-level systems (such as Kubernetes) exists a **middleware layer** that manages distributed communication.
This layer performs three important roles.

8.10.1 1. Remote Procedure Calls (RPC)

RPC allows one service to call a function running on another machine.
Steps involved:

1. Convert function parameters into a network message.
2. Send the message to the remote machine.
3. Execute the procedure remotely.
4. Return the result to the caller.

Analogy: Sending a gift by courier. The sender packages the gift, the courier transports it, and the receiver opens it.

8.10.2 2. Service Discovery

In distributed systems, services frequently start, stop, and move between machines.
Service Discovery acts as a dynamic directory that maps:

$$\text{Service Name} \rightarrow \text{Current Network Address}$$

This allows services to locate each other even when their IP addresses change.
Example:
The Order Service queries the service registry to locate the correct Restaurant Service instance.

8.10.3 3. Temporal Synchronization

In distributed systems, events occurring on different machines may not arrive in the same order due to network delays.

Temporal synchronization mechanisms help ensure that events are interpreted in the correct **causal order**.
This is important for operations such as:

- payments
- inventory updates
- order confirmations

8.11 The RPC Tax

Although RPC makes remote calls appear like local calls, they are fundamentally different.
Remote calls involve:

- network latency
- possible packet loss
- serialization and deserialization
- machine failures

This additional overhead is sometimes referred to as the **RPC Tax**.

8.12 Exam Insight

- RPC hides network complexity but does not remove it.
- Distributed systems rely on three key middleware functions:
    1. RPC (communication)
    2. Service Discovery (finding services)
    3. Temporal Synchronization (ordering events)

8.13 Syntactic Transparency and the RPC Illusion

Modern distributed applications are composed of many independent services running on different machines. Despite this complexity, developers often write code as if they are simply calling local functions.

This is possible due to **syntactic transparency** in Remote Procedure Calls (RPC).

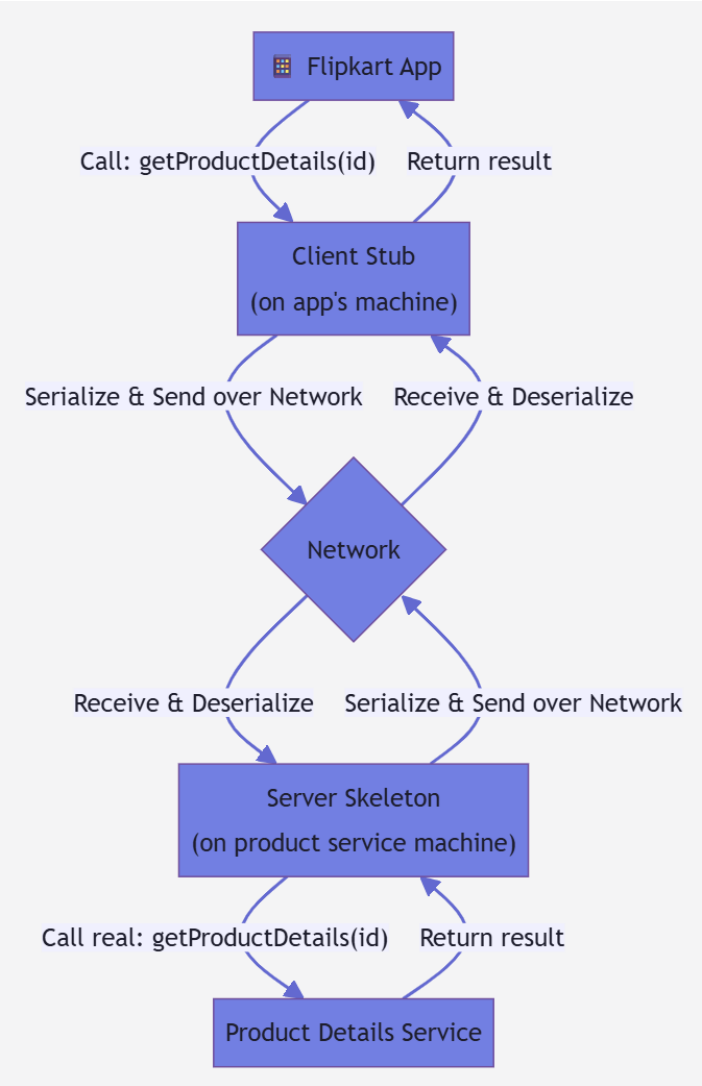
Definition

Syntactic transparency means that a remote call appears identical to a local function call from the programmer's perspective.

```
product = getProductDetails(productId)
```

Although this looks like a simple function call, the requested service may actually run on a different machine in a distributed data center.

8.13.1 Example: E-commerce Platform



Consider browsing products on an e-commerce platform.

When a user opens a product page, the application may need data from multiple services:

- Product Service → product description and images
- Pricing Service → price and discounts
- Review Service → ratings and customer feedback
- Recommendation Service → related products

Each service may run on a different machine. RPC allows these services to communicate while keeping the programming interface simple.

8.14 Client Stub and Server Skeleton

RPC systems use two main components to hide network communication.

8.14.1 Client Stub

The **client stub** acts as a proxy for the remote service.

Responsibilities:

- Accept parameters from the client program
- Serialize (marshal) the parameters
- Send the request across the network
- Receive the response
- Return the result to the caller

8.14.2 Server Skeleton

The **server skeleton** runs on the remote machine.

Responsibilities:

- Receive network requests
- Deserialize (unmarshal) the parameters
- Invoke the actual remote procedure
- Serialize the result
- Send the response back to the client

8.15 RPC Execution Steps

A remote call typically involves the following steps:

1. Client calls a function (appears local).
2. Client stub marshals the parameters.
3. Request is transmitted over the network.
4. Server skeleton unmarshals the parameters.
5. Remote procedure executes.
6. Result is serialized and returned.
7. Client stub delivers the result to the application.

9 Failure Semantics in RPC

In distributed systems, client and server communicate over a network. Unlike local procedure calls, RPC involves network delays, message loss, and server failures.

Because of these uncertainties, the client may not know whether a remote procedure was executed successfully.

To address this issue, RPC systems define **execution semantics** that specify how many times a procedure may execute.

9.1 At-Least-Once Semantics

Guarantee: The procedure executes one or more times.

If the client does not receive a response, it retries the request. This ensures that the operation eventually executes, but duplicate executions are possible.

This approach is suitable for **idempotent operations**, where repeating the operation produces the same result.

Example: file reads or database queries.

9.2 At-Most-Once Semantics

Guarantee: The procedure executes at most once.

The system prevents duplicate execution by detecting and suppressing repeated requests.

However, if a failure occurs before execution, the procedure may not execute at all.

This approach prevents duplication but risks data loss.

9.3 Exactly-Once Semantics

Guarantee: The procedure executes exactly once.

This is the ideal behavior for critical operations such as financial transactions.

Achieving exactly-once semantics requires:

- Stable storage logging
- Duplicate request detection

In practice, implementing exactly-once semantics in distributed systems is complex and expensive.

9.4 The RPC Tax

Although RPC makes remote calls appear like local calls, they are significantly more expensive.

A local function call typically takes **nanoseconds**, while a remote call may take **microseconds or milliseconds**.

This additional overhead is known as the **RPC Tax**.

Major sources of RPC overhead include:

- Serialization and deserialization of data
- Network transmission delays
- Memory copying between buffers
- CPU context switching during network operations

9.4.1 Sources of RPC Overhead

The main bottlenecks in RPC systems include:

- **Serialization (Marshalling)**
  Function arguments must be converted into a byte stream before being transmitted over the network.
- **Context Switching**
  RPC communication often requires switching between user space and kernel space, introducing additional latency.
- **Memory Copies**
  Data may be copied multiple times between application buffers, kernel buffers, and network interface buffers.

9.4.2 Hardware Acceleration

To reduce RPC overhead, specialized hardware solutions have been developed.

Arcalis

- RPC acceleration logic placed near the Last Level Cache (LLC)
- Reduces memory copying and communication latency

RDMA (Remote Direct Memory Access)

- Allows direct memory access between machines
- Bypasses the operating system kernel
- Reduces context switching and memory copies
- Provides significant performance improvements

10 RPC Tax in Deep Learning

Deep learning workloads involve large data structures called tensors. Tensors are multi-dimensional arrays used to store model weights and gradients.

Traditional RPC frameworks such as gRPC and Thrift are designed for small heterogeneous messages and are inefficient for transferring large homogeneous tensor data.

10.1 Communication Bottlenecks

Redundant Memory Copies

In traditional RPC communication, tensor data follows the path:

$$GPU \rightarrow CPU \rightarrow RPC\ Buffer \rightarrow Network$$

Each transfer introduces latency and CPU overhead. This phenomenon is often referred to as the **Triple Copy Problem**.

10.2 Parameter Server Architecture

Traditional distributed deep learning systems use the Parameter Server model.

- Worker nodes compute gradients.
- Parameter servers maintain global model weights.
- Workers send gradients to the server.
- The server aggregates gradients and sends updated weights back.

This architecture creates network congestion at the server node and introduces CPU–GPU communication overhead.

10.3 NCCL and Collective Communication

Modern systems use decentralized collective communication.

NCCL (NVIDIA Collective Communications Library) enables efficient communication between GPUs using collective primitives such as **AllReduce**.

AllReduce aggregates gradients across all nodes so that every GPU receives the final result.

NCCL is optimized for high-speed hardware interconnects such as NVLink and PCIe switches.

10.4 Real-World Impact

During large-scale events (e.g., major sales or flash events), millions of requests are generated simultaneously.

For example, when a user adds a product to their cart:

- Inventory service checks product availability
- Pricing service verifies the latest price
- Cart service updates the user's cart
- Recommendation service updates suggestions

Each of these steps may involve multiple RPC calls, and the accumulated RPC tax can significantly affect system performance.

10.5 Exam Insight

- RPC provides **syntactic transparency**.
- Remote calls appear identical to local function calls.
- However, remote calls incur significant overhead known as the **RPC Tax**.
- Client stubs and server skeletons hide the complexity of network communication.

10.6 Reliability and Performance Challenges in Distributed Systems

Distributed systems must deal with two major challenges:

- **Unreliable networks**
- **Performance overhead of remote communication**

Even though applications appear seamless to users, complex mechanisms operate behind the scenes to ensure correctness and efficiency.

10.6.1 Delivery Guarantees

When services communicate over a network, messages may be lost, duplicated, or delayed. Systems therefore provide different delivery guarantees.

- **At Most Once** – The request is processed at most once. If it fails, it is not retried.
- **At Least Once** – The request is retried until it succeeds, which may result in duplicate executions.
- **Exactly Once** – The operation is executed exactly one time. This is very difficult to guarantee in distributed systems.

Example

For applications such as social media feeds, duplicate or missed updates are usually acceptable, so weaker guarantees are sufficient.

However, financial systems such as digital payments require **exactly-once semantics** to ensure that money is neither deducted twice nor lost.

10.6.2 Achieving Exactly-Once Behavior

Because perfect exactly-once execution is extremely difficult in distributed systems, systems approximate it using:

- **Idempotent operations** – repeating the same request produces the same result.
- **Unique request identifiers** – used to detect duplicate requests.
- **Transaction logs** – track completed operations.

These mechanisms help prevent inconsistencies during failures or network delays.

10.6.3 RPC Overhead and the "RPC Tax"

Remote Procedure Calls make distributed communication appear like local function calls, but they introduce significant overhead.

Major sources of overhead include:

- Data serialization and deserialization
- Network transmission delays
- Memory copying between buffers
- CPU context switching and kernel processing

This overhead is commonly referred to as the **RPC Tax**.

10.6.4 Why AI Systems Avoid Standard RPC

Large-scale AI systems often need to transfer massive amounts of data (e.g., tensors) between GPUs. Traditional RPC mechanisms become inefficient due to their overhead.

To solve this, specialized technologies are used:

- **RDMA (Remote Direct Memory Access)**
- **GPUDirect**

These technologies allow network devices to directly transfer data between memory or GPUs without involving the CPU or operating system.

Key Benefit

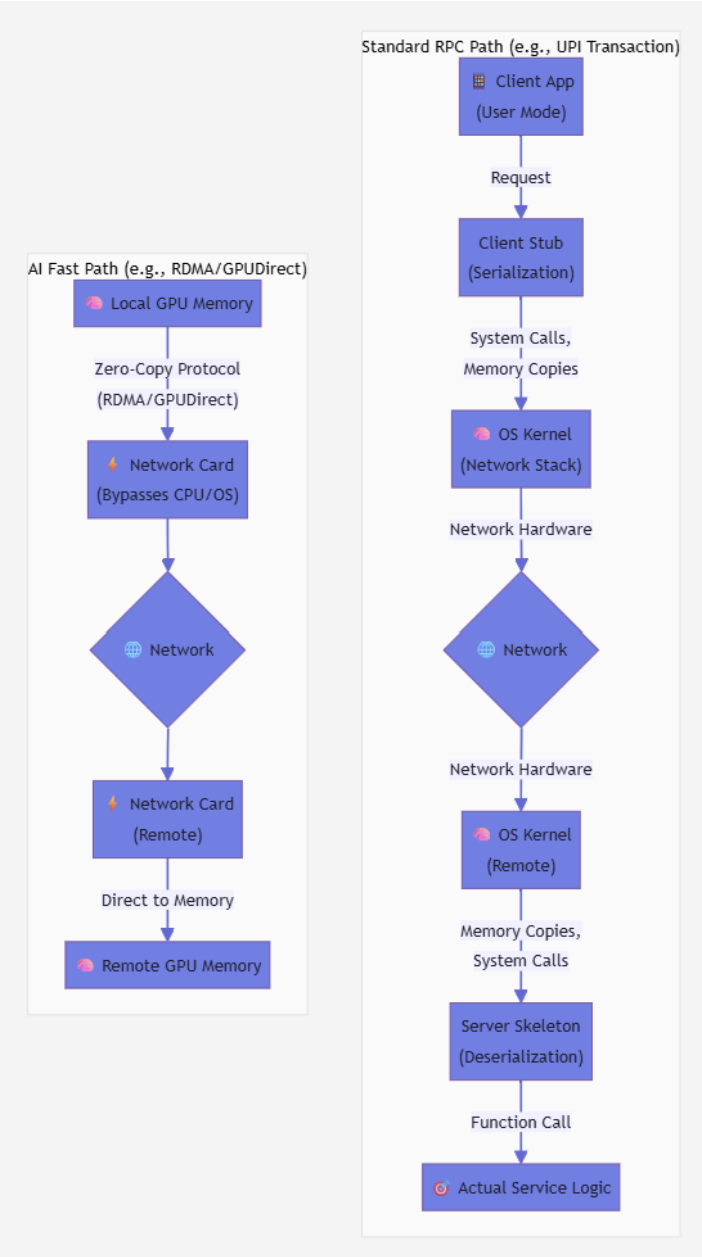
- Reduces data copying
- Minimizes latency
- Significantly increases throughput

10.6.5 Key Insight

Distributed systems must balance:

- **Correctness** (reliable message delivery and consistency)
- **Performance** (minimizing communication overhead)

Different applications require different trade-offs depending on their reliability and performance requirements.


Standard RPC Path (e.g., UPI Transaction)
Client App (User Mode)
Request
Client Stub (Serialization)
System Calls, Memory Copies
OS Kernel (Network Stack)
Network Hardware
Network
Network Hardware
OS Kernel (Remote)
Memory Copies, System Calls
Server Skeleton (Deserialization)
Function Call
Actual Service Logic
AI Fast Path (e.g., RDMA/GPUDirect)
Local GPU Memory
Zero-Copy Protocol (RDMA/GPUDirect)
Network Card (Bypasses CPU/OS)
Network
Network Card (Remote)
Direct to Memory
Remote GPU Memory

11 From Local OS to Cloud Operating System

11.1 1. Role of the Local Operating System

The Operating System (OS) acts as an intermediary between hardware and applications. Its main goals are:

- **Efficiency** – maximize hardware utilization
- **Fairness** – share resources among processes
- **Stability** – prevent system crashes

At the core of the OS is the **Kernel**, which runs in privileged mode and manages hardware resources.

11.1.1 User Mode vs Kernel Mode

- **User Mode (Ring 3)** – applications run here with restricted access.
- **Kernel Mode (Ring 0)** – the OS executes privileged operations.

Applications interact with hardware through **system calls**, which safely transition execution to kernel mode.

11.2 Hardware Abstraction

Linux simplifies hardware interaction through the philosophy:
"Everything is a File"
Examples:

Component	Linux Representation	Benefit
Hard Drive	/dev/sda	Storage accessed as a byte stream
Terminal	/dev/tty	Standardized input/output interface
Memory Info	/proc/meminfo	Real-time system monitoring

This abstraction allows standard tools to interact with different hardware components.

11.3 2. Process Lifecycle and Memory Management

A process contains multiple memory regions:

- **Text** – executable instructions
- **Data** – global variables
- **Stack** – function calls and local variables

Linux creates new processes using:

- **fork()** – duplicates the parent process
- **exec()** – loads a new program into the process

Optimization: Copy-on-Write (COW)
Parent and child initially share memory pages; duplication occurs only when modifications happen.

11.4 Process States

Common process states include:

- **TASK_RUNNING** – currently running or ready to run
- **TASK_INTERRUPTIBLE** – waiting for an external event
- **TASK_ZOMBIE** – finished execution but waiting for parent to collect exit status

11.5 Memory Efficiency

Linux uses **Demand Paging**, where memory pages are allocated only when accessed.

If memory becomes exhausted, the kernel activates the **OOM Killer** (Out of Memory Killer) to terminate low-priority processes and prevent system failure.

11.6 3. CPU Scheduling

Linux uses the **Completely Fair Scheduler (CFS)** to distribute CPU time fairly among processes.

It tracks the **virtual runtime (vruntime)** of each task and schedules the process that has used the least CPU time.

To manage runnable processes efficiently, CFS uses a **Red-Black Tree**, enabling:

- efficient task insertion and removal
- fast selection of the next process to run

11.7 4. Middleware in Distributed Systems

When systems expand from single machines to clusters, **middleware** abstracts network complexity.

Middleware provides transparency for:

- location of services
- migration of services
- replication of services
- failure handling

Feature	Local OS	Distributed Middleware
Scope	Single machine	Cluster-wide system
Abstraction	Processes, files	Services, messages
Communication	System calls	RPC / Message queues
Failures	System crash	Partial node failures

11.8 Example: RPC Request Flow

When a service calls another service in a distributed system:

1. Middleware marshals the request data.
2. The OS sends packets via the network stack.
3. The remote OS receives the packets.
4. Middleware unmarshals the data.
5. The remote service executes the request.

11.9 5. Cloud Operating Systems

Cloud platforms such as **Kubernetes** treat the entire data center as a single computer.

Architecture analogy:

Local OS Component	Kubernetes Equivalent
Kernel	Control Plane
Process	Pod
Device Driver	Kubelet

11.10 Desired State Model

Cloud systems operate using a **desired state model**:

1. Observe current system state
2. Compare with desired configuration
3. Take actions to correct the difference

Example: If the system requires 3 containers but only 2 are running, Kubernetes automatically launches another container.

11.11 Key Insight

- The **local OS** acts as the system's **execution engine (muscle)**.
- The **cloud OS** acts as the **global coordinator (brain)**.
- Together they enable large-scale distributed computing.

12 Distributed Transaction Management

In distributed systems, a single user operation may involve multiple independent services or databases (e.g., Order Service, Payment Service, Inventory Service). Transaction management ensures that such operations behave as a single unit of work with an **all-or-nothing** guarantee.

12.1 ACID Properties

Transactions follow the ACID properties:

- **Atomicity:** All operations succeed or the entire transaction rolls back.
- **Consistency:** The system moves from one valid state to another valid state.
- **Isolation:** Concurrent transactions behave as if executed sequentially.
- **Durability:** Once committed, changes persist even after failures.

12.2 Challenge in Microservices

In microservice architectures, services run independently and may fail separately. For example, payment may succeed while inventory update fails, leading to inconsistent system states.

12.3 Two-Phase Commit (2PC)

Two-Phase Commit is a protocol used to ensure atomic transactions across multiple services.

Phase 1: Prepare Phase

- Coordinator asks all participants if they can commit.
- Participants lock resources and vote `YES` or `NO`.

Phase 2: Commit / Abort Phase

- If all participants vote YES, the coordinator sends `COMMIT`.
- If any participant votes NO, the coordinator sends `ABORT`.

12.4 Limitations of 2PC

- Blocking if the coordinator fails.
- Reduced scalability due to resource locking.
- Additional network communication overhead.

12.5 Example: Uber Ride Booking

Booking a ride involves multiple services such as driver assignment and payment processing. If a driver is assigned but payment fails, the ride must be cancelled. If payment succeeds but no driver is found, the system must issue a refund. Such systems often use compensating actions (e.g., refunds) to restore consistency.

13 Distributed Transaction Management

In distributed systems, a single user operation may involve multiple independent services (e.g., Payment, Inventory, Order). Ensuring consistency across these services is difficult due to network failures, service crashes, and partial execution.

Two common approaches to handle distributed transactions are:

- Two-Phase Commit (2PC)
- Saga Pattern

14 Two-Phase Commit (2PC)

Two-Phase Commit is a protocol used to guarantee **atomicity** across multiple distributed participants.

14.1 Participants

- **Coordinator (Transaction Manager)** – manages the transaction
- **Participants (Resource Managers)** – databases or services

14.2 Phase 1: Prepare (Voting Phase)

- Coordinator sends a `prepare` request to all participants.
- Each participant performs its local work and locks resources.
- Participants vote `YES` or `NO`.
- If any participant votes `NO`, the transaction will abort.

14.3 Phase 2: Commit / Abort

- If all participants vote YES → coordinator sends `COMMIT`.
- If any participant votes NO → coordinator sends `ABORT`.
- Participants finalize or rollback their local transactions.

14.4 Advantages

- Strong atomicity (all-or-nothing guarantee)
- Ensures global consistency across services

14.5 Limitations

- Blocking protocol – participants hold locks during the transaction
- Coordinator is a single point of failure
- Poor scalability in large distributed systems
- Multiple network round trips increase latency

15 Saga Pattern

The Saga Pattern is an alternative approach for distributed transactions in microservices architectures.

Instead of locking resources globally, a distributed transaction is broken into a sequence of **local transactions**. Each step has a corresponding **compensating transaction** that can undo the change if a failure occurs.

Sagas prioritize **availability and scalability** and rely on **eventual consistency**.

15.1 Key Concepts

- **Local Transaction** – each service commits its own database update.
- **Compensating Transaction** – reverses the effects of a previous step.
- **Saga Log** – persistent record tracking saga progress.
- **Eventual Consistency** – the system becomes consistent after all steps complete.

15.2 Saga Execution Models

1. Orchestration

- A central orchestrator manages the workflow.
- The orchestrator invokes each service sequentially.
- If a step fails, compensating transactions are triggered.

Example flow:

1. Payment Service processes payment.
2. Inventory Service reserves stock.
3. Delivery Service assigns delivery.

If delivery assignment fails:

- Cancel inventory reservation
- Refund payment

2. Choreography

- No central coordinator.
- Services communicate using events.
- Each service reacts to events and triggers the next step.

Example event flow:

- `OrderPlaced` → Payment Service processes payment
- `PaymentProcessed` → Inventory Service reserves stock
- `InventoryReserved` → Shipping Service prepares shipment

If inventory fails:

- `InventoryFailed` event triggers payment refund

16 Comparison: Saga Pattern vs Two-Phase Commit

Aspect	Two-Phase Commit (2PC)	Saga Pattern
Consistency Model	Strong atomicity	Eventual consistency
Availability	Lower (blocking)	Higher (non-blocking)
Scalability	Limited	Highly scalable
Coordinator	Required	Optional or none
Failure Handling	Abort transaction	Compensating actions
Isolation	Global locking	Local transactions only
Typical Use Case	Financial systems	Microservices workflows

17 Example: UPI Payment Transaction

A typical UPI transaction involves multiple distributed components.

1. Sender bank debits the amount and marks transaction as pending.
2. Payment is routed through the NPCI network.
3. Receiver bank credits the amount.
4. Both users receive confirmation notifications.

Failure Handling (Saga Compensation)

- If the network transfer fails → sender bank reverses the debit.
- If the receiver bank is unavailable → transaction times out and refund occurs.

This demonstrates eventual consistency using compensating transactions.

18 Key Idea

Two-Phase Commit ensures strong atomicity but suffers from blocking and scalability limitations. The Saga Pattern decomposes transactions into local steps with compensating actions, enabling high availability and scalability in modern microservices systems.

19 Observability in Distributed Systems

Observability is the ability to infer a system's internal state from its external outputs. It is essential in distributed systems where failures and performance issues may occur across many independent services.

19.1 Three Pillars of Observability (MELT)

- **Metrics** – Aggregated numerical measurements over time (e.g., latency, throughput, CPU usage). Tools: Prometheus, Atlas.
- **Events / Logs** – Timestamped records describing events or system behavior. Tools: ELK Stack, Loki, Fluentd.
- **Traces** – End-to-end tracking of a request across multiple services to analyze dependencies and latency. Tools: Jaeger, Zipkin, OpenTelemetry.

19.2 Benefits

- Faster debugging and root-cause analysis
- Identification of performance bottlenecks
- Improved reliability through monitoring and alerts
- Audit trails for security and compliance

19.3 Challenges

- Correlating data across many services
- High storage cost due to large telemetry data
- Privacy and data management concerns

19.4 Common Observability Stack

Prometheus collects metrics, Grafana visualizes dashboards, and Jaeger/Zipkin trace distributed requests. Logging systems such as ELK or Loki aggregate logs for analysis.

19.5 Relation with Journaling

Observability also monitors storage systems:

- WAL append latency recorded as metrics
- Log compaction events stored as logs
- Recovery and replay operations traced

Key Idea: Observability helps engineers detect, diagnose, and resolve failures in large distributed systems.

20 Journaling and Checkpointing in Distributed Systems

20.1 Journaling / Write-Ahead Logging (WAL)

Journaling records all changes in an append-only log before applying them to the main data structures.

- Ensures **durability**: acknowledged writes survive crashes.
- Enables **crash recovery** by replaying the log.
- Supports **atomicity** in transactions.
- Logs are often **replicated across nodes** using protocols like Raft or Paxos.

20.2 Checkpointing

Checkpointing periodically creates a consistent snapshot of the system state (memory, metadata, indexes).

- Reduces recovery time by avoiding replay of the entire log.
- Allows **log truncation / compaction** after a checkpoint.
- Trade-off: checkpoint creation may cause temporary I/O overhead.

20.3 Integration of Journaling and Checkpointing

1. All updates are first appended to the WAL.
2. The system periodically creates a checkpoint (state snapshot).
3. Logs prior to the checkpoint are removed or compacted.
4. During recovery: load checkpoint + replay remaining logs.

20.4 Benefits

- Crash recovery and durability
- Faster recovery using checkpoints
- Bounded storage through log compaction
- Supports exactly-once semantics in distributed systems

20.5 Challenges

- Disk overhead for logs
- Checkpoint creation can temporarily slow writes
- Long logs increase replay time if checkpoints are infrequent

20.6 Real-World Examples

- **Redis**: Append-Only File (AOF) for journaling, log rewrite for compaction.
- **Kafka**: Immutable commit logs replicated across brokers.
- **HDFS**: Edit logs + periodic fsimage checkpoints.
- **etcd**: Raft WAL with periodic snapshots.
- **RocksDB**: WAL + memtable flushes acting as checkpoints.
- **ZooKeeper**: Transaction logs with periodic snapshots.

Key Idea: Journaling guarantees durability through logging, while checkpointing improves recovery speed and limits log growth.

Aspect	**Observability**	**Journaling**
Purpose	Monitoring and debugging systems	Data durability and crash recovery
Data Type	Metrics, logs, traces (telemetry)	Transaction logs (WAL)
Used By	Engineers / operators	Databases / storage systems
Goal	Understand system behavior	Recover system state after failures
Examples	Prometheus, Grafana, Jaeger	PostgreSQL WAL, Kafka logs

Table 1: Observability vs Journaling

21 Queuing Theory

Queuing theory provides a mathematical framework for analyzing system performance under variable load. It models request arrivals, service processes, and server capacity to predict bottlenecks and guide infrastructure scaling.

21.1 Kendall's Notation

Queuing systems are commonly described using Kendall's notation ($A/S/c$):

- **A (Arrival Process)** – distribution of incoming requests. For Markovian systems (M), arrivals follow a Poisson process.
- **S (Service Time)** – distribution of service times. For Markovian systems (M), service times follow an exponential distribution.
- **c (Servers)** – number of parallel servers handling requests.

21.2 Birth–Death Process

System state transitions are modeled using a birth–death process:

- **Birth rate** (λ) – arrival rate of requests.
- **Death rate** (μ) – service completion rate.

At equilibrium, the rate entering a state equals the rate leaving it.

21.3 M/M/1 Queue

The M/M/1 model represents a system with:

- Markovian arrivals
- Markovian service times
- One server

System utilization is defined as:

$$\rho = \frac{\lambda}{\mu}$$

Using Little's Law:

$$L = \lambda W$$

where L is the average number of requests in the system and W is the average response time.

21.4 Key Insight

As utilization $\rho \to 1$, response time increases rapidly (the "hockey-stick" effect). This means single-server systems (M/M/1) become unstable under heavy load, making horizontal scaling to multi-server systems (M/M/c) necessary.

21.5 M/M/c Queue (Multi-Server Systems)

The M/M/c model represents a queue with:

- Markovian arrivals (Poisson)
- Markovian service times (Exponential)
- c parallel servers

Unlike M/M/1, multiple servers process requests simultaneously, increasing system capacity and improving performance under high load.

System utilization is defined as:

$$\rho = \frac{\lambda}{c\mu}$$

where:

- λ = arrival rate
- μ = service rate per server
- c = number of servers

Increasing the number of servers allows the system to handle higher arrival rates while maintaining stability.

21.6 Advantages of M/M/c over M/M/1

- **Higher Capacity:** Total service capacity scales with $c\mu$.
- **Lower Waiting Time:** Multiple servers reduce queue delays.
- **Better Scalability:** System can handle traffic bursts.
- **Lower Saturation Risk:** Less chance of hitting the response-time explosion seen in M/M/1 systems.

21.7 M/M/c Queue Formulas

Probability of an Empty System

$$P_0 = \left[\sum_{n=0}^{c-1} \frac{(c\rho)^n}{n!} + \frac{(c\rho)^c}{c!(1-\rho)} \right]^{-1}$$

This represents the probability that the system has fewer requests than available servers (i.e., no queue forms).

Total Response Time

$$W = \frac{P_0 (c\rho)^c \rho}{c!(1-\rho)^2 \lambda} + \frac{1}{\mu}$$

where

$$\rho = \frac{\lambda}{c\mu}$$

- λ = arrival rate
- μ = service rate per server
- c = number of servers
- ρ = system utilization

Key Insight: Multi-server queues (M/M/c) provide better scalability and stability than single-server queues (M/M/1), making them suitable for high-traffic distributed systems.

21.8 Pooling Effect

In M/M/c systems, servers share a single queue. This pooling of resources reduces the probability that all servers are busy simultaneously and improves overall system efficiency.